Self-Consistent Proportional Per-Node Damping for Reachability Ranking in Uncertain Graphs

Technical documentation: architecture, proofs, code walkthrough, complexity and reproduction guide

Ioannis Konstantellos
Computer Engineering and Informatics Department
University of Patras, Greece

October 2026

Abstract

This document accompanies the paper *Self-Consistent Proportional Per-Node Damping for Reachability Ranking in Uncertain Graphs: Exact Theory, Dynamical Obstructions, and Hybrid Dual-Operator Polishing* and its Greek version. It describes the architecture of the software package that produced every number in the paper, gives complete proofs of Theorem 1 and of Propositions A–F and $2'$ together with the auxiliary lemmas on which the implementation relies, walks through every module, analyses the time and memory complexity of each component, and gives a step-by-step guide for reproducing the experiments, the tables, the figures and the four PDF documents from scratch.

Contents

1 Scope and conventions **2**

2 Architecture **2**
2.1 Repository layout . . . 2
2.2 Data flow . . . 3
2.3 Determinism . . . 3
2.4 File formats . . . 3

3 Mathematical foundations **4**
3.1 Auxiliary results used by the implementation . . . 5
3.2 Theorem 1 . . . 5
3.3 Collapse, absorption and instability . . . 6
3.4 Realization C . . . 7
3.5 The sign obstruction . . . 7
3.6 The pull variant . . . 8
3.7 Prior dispersion . . . 8

4 Module-by-module walkthrough **8**
4.1 `prop_damping_lib.py` . . . 8
4.2 `build_panels.py` . . . 9
4.3 `experiments.py` . . . 10
4.4 `select_configs.py` . . . 10
4.5 Analysis, controls and diagnostics . . . 10
4.6 Reporting . . . 10
4.7 Tests . . . 11

5 Complexity analysis **11**

6 Reproduction guide **12**
6.1 Environment 12
6.2 Step by step 12
6.3 From the paper to the code 13

7 Extending the package **14**

1 Scope and conventions

The package studies the Proportional Relative Influence Principle for per-vertex damping in diffusion-based reachability ranking. It contains (i) a vectorised library implementing uncertain-graph instances, Monte Carlo ground truth, propagation operators, damped diffusions, message passing, the three realizations of the principle, the anchored power family, Mode (b) polishing, oracles and metrics; (ii) the experiment runners for a pre-registered evaluation on 688 instances; (iii) analysis, control and benchmarking scripts; and (iv) the generators of every macro, table and figure of the paper.

Notation follows the paper. $G = (V, E, p)$ is an undirected uncertain graph with $n = |V|$ vertices and $m = |E|$ edges, each present independently with probability $p_{ij} \in (0, 1]$; $x > 0$ with $\sum_i x_i = 1$ is the prior; $y_i^* = \mathbb{E}\left[\sum_{v \in C(i)} x_v\right]$ is the expected reachable mass. Vectors are compared componentwise, $\circ$ is the componentwise product, $\mathscr{N}(v) = v/\mathbf{1}^\top v$ and $\mathbf{1}$ is the all-ones vector. In file names, `<panel>` is one of `dev`, `test`, `prior`, `real`, `scale`. In result files a step count $T = 10^6$ stands for "run to the fixed point" ($T = \infty$).

2 Architecture

2.1 Repository layout

Table 1 lists the files and directories of the package.

Table 1: Repository layout.

Path	Content
`prop_damping_lib.py`	The library: instances, priors, real-network loading, operators, Monte Carlo and exact ground truth, metrics, diffusions, message passing, the realizations, the anchored law, polishing, oracles.
`build_panels.py`	Generates the five panels and their two independent ground-truth samples.
`experiments.py`	The `grid` stage (full search on dev and test) and the `confirm` stage (references, oracles and the frozen configurations on the held-out panels).
`select_configs.py`	Re-derives the frozen configurations from the development results by the stated rules and verifies `results/selected.json`.
`analysis.py`	Loading of stage results, configuration means, paired statistics, Holm correction.
`controls.py`, `mf_controls.py`	Matched controls: constant damping, per-step mean (mean-field twin) and permutation of the per-node law.
`sign_condition.py`, `prop2_stats.py`	Diagnostics for Proposition E (sign condition) and Proposition 2 (prior uniformity).
`explore_rawP.py`	Exploratory (not pre-registered) raw-P propagation.
`mc_baseline.py`, `bench_runtime.py`	Monte Carlo baselines at $k \in \{50, 200, 1000\}$ worlds and single-process timings.
`paper_extras.py`	Dynamics traces, operator complementarity and the size sweep.

Table 1: Repository layout (continued).

Path	Content
`make_checkpoint_tables.py`, `write_checkpoint_report.py`	The tables and report of the first checkpoint.
`make_paper_assets.py`, `paper_figures.py`	Every macro, table and figure of the paper (English and Greek).
`tests/`	Foundational and dynamical checks (Theorem 1, ground truth, propositions).
`data/<panel>/NNNN.npz`	Instances with their ground truth; `data/<panel>/specs.json` lists the specifications.
`results/`	Per-instance results, controls, diagnostics, timings, the frozen selection and its digest.
`paper/`	`paper_en.tex`, `paper_gr.tex`, the generated assets in `paper/generated/` and the compiled PDFs.
`figures/`, `figures_gr/`	The five figures with English and Greek labels.
`logs/`	Progress logs of the long stages (used for the run times reported here).

2.2 Data flow

Figure 1 shows the pipeline. Each stage reads only the outputs of earlier stages, so any stage can be rerun in isolation. Selection uses the development panel only, and every held-out stage reads the frozen configuration file.

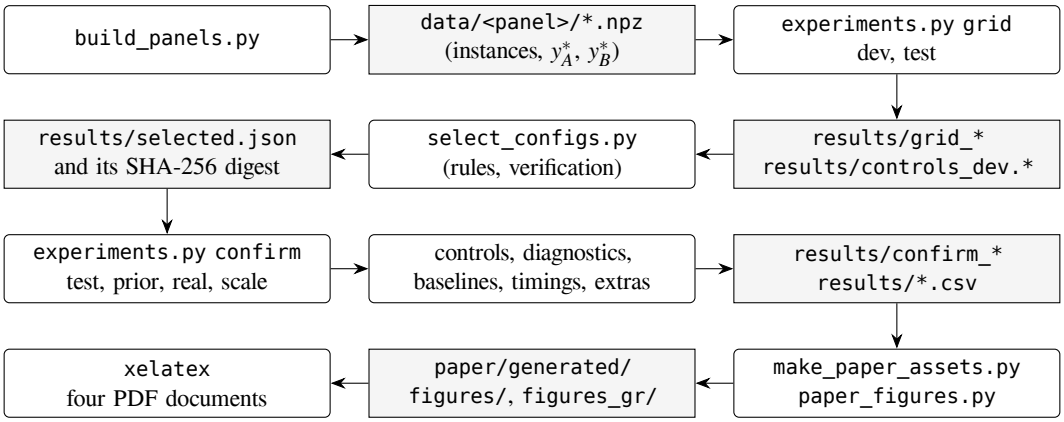


Figure 1: Data flow of the package. Grey boxes are files; white boxes are programs.

2.3 Determinism

Every random draw is seeded by `stable_seed(*parts)`, the first 63 bits of the SHA-256 digest of the role of the draw (for instance the panel, family, size, probability range and replicate of an instance, or the instance name and the purpose of a Monte Carlo sample). Seeds therefore do not depend on the order of execution or on the number of worker processes, and rerunning any stage reproduces the stored results bit for bit on the same platform. Topology and edge probabilities of a synthetic instance are seeded independently of its prior, which is what allows the prior panel to vary only the prior.

2.4 File formats

Instances. `data/<panel>/NNNN.npz` stores `edges` (an $m \times 2$ integer array with $i < j$), `p` (edge probabilities), `x` (prior), `yA` and `yB` (two independent 2,000-world ground-truth estimates), `yA2` (the second moment of

the evaluation sample) and `meta` (a JSON string with family, size, probability range, prior, replicate, $\bar{P}_{\text{esc}}$, prior statistics and the split-sample reference). `load_instance` returns the instance, `yA`, `yB` and the metadata.

Results. Each stage writes `results/<stage>_<panel>/<id>.csv.gz` with one row per evaluated method or configuration and the columns `id`, `method`, `T` (steps or sweeps; 0 for single-shot methods), `beta` (blend weight), `blend` (`s` score blend, `r` rank blend, `-` none) and `rho` (Spearman against y_A^*), plus a JSON file with diagnostics (oracle fits, dynamics, secondary metrics).

Method names. The prefix of a method name identifies its family; the fields after the colon are separated by vertical bars.

Prefix or pattern	Meaning
`base:`	reference diffusions: `rule_EW`, `fixed085`, `rule_WT`, `local_pesc`
`ref:`	references: `prior`, `Pesc`, `MP2`, `MPinf`, `ceiling`
`oracle:`	oracles: `scalar`, `scalar_in`, `scalar_WT`, `pernode`
`true:`	the per-node laws evaluated with the true y^*: `prop`, `ap0.5`, `ap1.0`
`sa:kind|op|scale|gamma`	standalone realizations
`ap:eta`	the anchored law
`free:gamma|eta`	the unanchored law
`pb:base|eta|mode`	anchored polishing (`fix`: fixed α; `sc`: self-consistent)
`pbx:base|kind|op|scale|gamma`	realization polishing
`pbperm:base`	permuted anchored polishing
`sel:NAME`	a frozen configuration

Frozen configurations. `results/selected.json` maps each of the 21 names to its parameters (Table 2); `results/selected.sha256` records its digest (beginning with `b55fe19a10d3e5de`) and the time it was frozen (2026-10-25 22:41).

Table 2: The frozen configurations (`results/selected.json`).

Key	Type	Parameters
`A_best`	sa	`kind=A,op=P,scale=ex,gamma=16.0,T=2`
`B_best`	sa	`kind=B,op=EP,scale=ex,gamma=16.0,T=2`
`C_best`	sa	`kind=C,op=EP,scale=ex,gamma=16.0,T=2`
`C_dir1`	sa	`kind=C,op=P,scale=raw,gamma=1.0,T=3`
`B_unif`	sa	`kind=B,op=WT,scale=ex,gamma=16.0,T=50`
`AP_eta0.25`	ap	`eta=0.25,T=inf`
`AP_eta1`	ap	`eta=1.0,T=inf`
`PB_rule_best`	pb	`base=rule,eta=-0.25,mode=fix,T=1,beta=0.25,blend=s`
`PB_rule_nominal`	pb	`base=rule,eta=1.0,mode=fix,T=1,beta=0.5,blend=s`
`PB_rule_eta1`	pb	`base=rule,eta=1.0,mode=sc,T=inf,beta=0.25,blend=r`
`PB_mp2_best`	pb	`base=mp2,eta=0.25,mode=sc,T=5,beta=0.75,blend=r`
`PB_mp2_eta0`	pb	`base=mp2,eta=0.0,mode=sc,T=5,beta=0.75,blend=r`
`PB_mp2_eta1`	pb	`base=mp2,eta=1.0,mode=sc,T=5,beta=0.75,blend=r`
`PBX_rule_best`	pbx	`base=rule,kind=C,op=P,scale=raw,gamma=4.0,T=5,beta=0.5,blend=s`
`PBX_rule_unif`	pbx	`base=rule,kind=A,op=P,scale=raw,gamma=1.0,T=1,beta=0.5,blend=s`
`PBX_rule_dir1`	pbx	`base=rule,kind=C,op=P,scale=raw,gamma=1.0,T=5,beta=0.5,blend=r`
`PBX_rule_nominalB`	pbx	`base=rule,kind=B,op=EW,scale=ex,gamma=1.0,T=1,beta=0.5,blend=s`
`CTRL_Cmean`	ctrl	`law=Cmean,gamma=4.0,T=5,beta=0.5,blend=s`
`CTRL_Cperm`	ctrl	`law=Cperm,gamma=4.0,T=5,beta=0.5,blend=s`
`CTRL_lin0.5`	ctrl	`law=lin0.5,gamma=4.0,T=5,beta=0.5,blend=s`
`CTRL_lin0.9`	ctrl	`law=lin0.9,gamma=1.0,T=5,beta=0.5,blend=r`

3 Mathematical foundations

This section collects every mathematical statement used by the paper or by the implementation, with complete proofs. Throughout, $T \geq 0$ entrywise; T is *row-substochastic* if $T\mathbf{1} \leq \mathbf{1}$; $\rho(\cdot)$ denotes the spectral radius.

3.1 Auxiliary results used by the implementation

Lemma 1 (normalised damped diffusion)**.** *Let $\alpha \in [0,1]^n$, $D = \mathrm{diag}(\alpha)$ and $B(\alpha) = (I-D)\,x\mathbf{1}^\top + DT$. A vector $y > 0$ with $\mathbf{1}^\top y = 1$ is a fixed point of $y \mapsto \mathcal{N}\big((1-\alpha)\circ x + \alpha \circ Ty\big)$ if and only if it is an eigenvector of $B(\alpha)$ with eigenvalue $\lambda = \mathbf{1}^\top B(\alpha)y > 0$; it then satisfies $\lambda y = (I-D)x + DTy$. If $\alpha_i < 1$ for every i, then $B(\alpha) > 0$ entrywise, the eigenvector is unique (the Perron vector), and both the plain and the lazy power iterations converge to it from every positive start. If $B(\alpha)$ is only irreducible (for instance $\alpha \equiv 1$ and T irreducible but periodic), the lazy iteration $y \leftarrow \mathcal{N}\big(\frac{1}{2}(y + B(\alpha)y)\big)$ still converges to the Perron vector.*

Proof. For $\mathbf{1}^\top y = 1$ we have $B(\alpha)y = (I-D)x + DTy$, so the fixed-point equation $y = \mathcal{N}(B(\alpha)y)$ is $B(\alpha)y = (\mathbf{1}^\top B(\alpha)y)\,y$. If every $\alpha_i < 1$, the rank-one term $(I-D)x\mathbf{1}^\top$ is entrywise positive, so $B(\alpha) > 0$ and Perron's theorem [3] gives a simple, strictly dominant eigenvalue with a positive eigenvector, the only positive eigenvector up to scaling; the power method converges to it from every positive start. The lazy matrix $\frac{1}{2}(I+B)$ has the same eigenvectors and eigenvalues $\frac{1}{2}(1+\mu)$. If B is irreducible with Perron root r, every other eigenvalue satisfies $|\frac{1}{2}(1+\mu)| \le \frac{1}{2}(1+|\mu|) \le \frac{1}{2}(1+r)$, with equality only when $\mu = r$; hence $\frac{1}{2}(1+r)$ is strictly dominant, and the lazy power method converges even when B has other eigenvalues of modulus r (periodic case). □

Lemma 2 (anchoring constant)**.** *For real numbers $\ell_1, \dots, \ell_n$ and $\alpha_0 \in (0,1)$ there is a unique $c \in \mathbb{R}$ with $\frac{1}{n}\sum_i \sigma(c+\ell_i) = \alpha_0$, where σ is the logistic function. A Newton iteration safeguarded by the bracket $[\mathrm{logit}(\alpha_{\min}) - \max|\ell| - 1,\ \mathrm{logit}(\alpha_{\max}) + \max|\ell| + 1]$ and by bisection converges to it.*

Proof. $h(c) = \frac{1}{n}\sum_i \sigma(c+\ell_i)$ is continuous and strictly increasing, with $h(c) \to 0$ as $c \to -\infty$ and $h(c) \to 1$ as $c \to \infty$, so $h(c) = \alpha_0$ has exactly one solution, and it lies in the bracket because h at the endpoints is below $\alpha_{\min} \le \alpha_0$ and above $\alpha_{\max} \ge \alpha_0$. The implementation keeps the bracket updated by the sign of $h(c) - \alpha_0$ and replaces any Newton step that leaves it by the midpoint, so the bracket shrinks at every bisection; h is smooth and convex–concave, and Newton converges locally quadratically. □

Lemma 3 (batched Monte Carlo)**.** *Let $G'_1, \dots, G'_b$ be independent worlds and let H be their disjoint union, the graph on bn vertices whose block w contains the edges of G'_w. The connected components of H are exactly the components of the individual worlds. Consequently the estimator $\hat{y}_i = \frac{1}{M}\sum_{w=1}^{M}\sum_{v \in C_w(i)} x_v$ computed batch by batch is unbiased for y^*_i, with variance $\mathrm{Var}\big(\sum_{v\in C(i)} x_v\big)/M$, which the second moment stored in `yA2` estimates.*

Proof. H has no edge between different blocks, so every path, and hence every component, lies in one block, where it coincides with a component of the corresponding world. The summands $\sum_{v\in C_w(i)} x_v$ are independent copies of $\sum_{v\in C(i)} x_v$; linearity of expectation and independence give the mean and the variance. □

Lemma 4 (message passing at zero iterations)**.** *In the membership form of the message passing of [2], initialised with all messages equal to one, the vertex membership after zero iterations is $\mu_i = 1 - \prod_{j\in N(i)}(1-p_{ij}) = P_{\mathrm{esc}}(i)$.*

Proof. The vertex output is $\mu_i = 1 - \prod_{j\in N(i)}(1 - p_{ij}\nu_{j\to i})$, and with $\nu \equiv 1$ this is the escape probability. The implementation evaluates the product in log space, $\mu_i = -\mathrm{expm1}\big(\sum_j \mathrm{log1p}(-p_{ij}\nu_{j\to i})\big)$, which agrees with P_{esc} to within 1.1×10^{-16} in the tests. □

Lemma 5 (excess embedding)**.** *For $y^{\mathrm{base}} > 0$ and $\gamma > 0$ let $z = \mathcal{N}(y^{\mathrm{base}})/\big(\frac{1}{n}\sum_j \mathcal{N}(y^{\mathrm{base}})_j/x_j\big)$ and $y^{(0)} = x + \gamma z$. Then $y^{(0)} > x$, the excess odds $(y^{(0)}_i - x_i)/x_i$ have mean exactly γ, and the ranking of the excess $y^{(0)} - x$ is that of y^{base}/x.*

Proof. $z > 0$ and $\frac{1}{n}\sum_i z_i/x_i = 1$ by construction, so $\frac{1}{n}\sum_i (y^{(0)}_i - x_i)/x_i = \gamma$. The excess odds $\gamma z_i/x_i$ are a positive multiple of y^{base}_i/x_i. □

3.2 Theorem 1

Theorem 1 (exact solution of the coupled system)**.** *Let $x_i > 0$ and $S_i \ge 0$. A pair (y_i, α_i) with $y_i > 0$ and $\alpha_i \ne 1$ solves*

$$y_i = (1-\alpha_i)x_i + \alpha_i S_i, \qquad \frac{\alpha_i}{1-\alpha_i} = \frac{y_i - x_i}{x_i}$$

if and only if $\alpha_i = 1 - x_i/y_i$ and $(y_i - x_i)(y_i + x_i - S_i) = 0$. The solutions are the trivial branch $(x_i, 0)$ and, when $S_i > x_i$, the active branch $y_i = S_i - x_i$, $\alpha_i = (S_i - 2x_i)/(S_i - x_i)$, which has $\alpha_i \in (0,1)$ if and only if $S_i > 2x_i$. The quadratic $y_i^2 - S_i y_i + x_i S_i - x_i^2$ has discriminant $(S_i - 2x_i)^2$, so the branches meet exactly at $S_i = 2x_i$.

Proof. Step 1. If $\alpha_i \neq 1$, the proportionality equation is equivalent to $\frac{\alpha_i}{1-\alpha_i} + 1 = \frac{y_i}{x_i}$, i.e. $\frac{1}{1-\alpha_i} = \frac{y_i}{x_i}$, i.e. $\alpha_i = 1 - \frac{x_i}{y_i}$ (using $y_i \neq 0$). *Step 2.* Substituting $1 - \alpha_i = x_i/y_i$ and $\alpha_i = (y_i - x_i)/y_i$ in the diffusion equation and multiplying by $y_i > 0$ gives the equivalent equation $y_i^2 = x_i^2 + (y_i - x_i)S_i$. *Step 3.* Rearranging, $(y_i^2 - x_i^2) - S_i(y_i - x_i) = 0$, and factoring $y_i^2 - x_i^2 = (y_i - x_i)(y_i + x_i)$ gives $(y_i - x_i)(y_i + x_i - S_i) = 0$. Expanded, this is $y_i^2 - S_i y_i + x_i S_i - x_i^2 = 0$, whose discriminant is $S_i^2 - 4(x_i S_i - x_i^2) = (S_i - 2x_i)^2$ and whose roots are $\frac{1}{2}\big(S_i \pm |S_i - 2x_i|\big)$, i.e. x_i and $S_i - x_i$. *Step 4.* The root $y_i = x_i$ gives $\alpha_i = 0$. The root $y_i = S_i - x_i$ is admissible ($y_i > 0$) if and only if $S_i > x_i$, and then $\alpha_i = 1 - x_i/(S_i - x_i) = (S_i - 2x_i)/(S_i - x_i)$; its denominator is positive, so $\alpha_i > 0$ if and only if $S_i > 2x_i$, and $\alpha_i < 1$ always. *Verification.* With $y_i = S_i - x_i$ and $\alpha_i = (S_i - 2x_i)/(S_i - x_i)$: $(1 - \alpha_i)x_i + \alpha_i S_i = \big(x_i^2 + S_i^2 - 2x_i S_i\big)/(S_i - x_i) = S_i - x_i$, and $\alpha_i/(1 - \alpha_i) = (S_i - 2x_i)/x_i = (y_i - x_i)/x_i$. □

Remark 1 (the operator scale)**.** At the target, the active branch requires $S = Ty^* = y^* + x$. Since $y^* > 0$ and $(Ty^*)_i/y_i^* = 1 + x_i/y_i^* > 1$ for every i, the Collatz–Wielandt bound (Lemma 6) gives $\rho(T) > 1$. Hence no operator with $\rho(T) \leq 1$ — in particular no row-substochastic operator such as W or EW — can satisfy the coupled system at the target on the active branch.

Lemma 6 (Collatz–Wielandt lower bound [3, Thm. 8.1.26])**.** *If $A \geq 0$, $v > 0$ and $Av \geq cv$ for a real c, then $\rho(A) \geq c$.*

3.3 Collapse, absorption and instability

Proposition A (finite-time collapse of Realization A)**.** *Let T be row-substochastic, $x = \frac{1}{n}\mathbf{1}$, and $y^{(t+1)} = \max\big(x, Ty^{(t)} - x\big)$ with $y^{(0)} \geq x$. Then $M_t = \max_i y_i^{(t)}$ satisfies $M_{t+1} \leq \max\big(\frac{1}{n}, M_t - \frac{1}{n}\big)$ and $y^{(t)} = x$ for all $t \geq \lceil nM_0 \rceil - 1$; from $y^{(0)} = x + Tx$ the collapse occurs at $t = 1$.*

Proof. Invariance. $y^{(t)} \geq x$ for all t: true at $t = 0$ and preserved by the maximum. *Contraction of the maximum.* Since $T \geq 0$, $(Ty)_i \leq \sum_j T_{ij} \max_k y_k \leq M_t$ because the row sums are at most one. Therefore $y_i^{(t+1)} = \max(\frac{1}{n}, (Ty^{(t)})_i - \frac{1}{n}) \leq \max(\frac{1}{n}, M_t - \frac{1}{n})$ for every i. *Induction.* $M_t \leq \max(\frac{1}{n}, M_0 - \frac{t}{n})$; once $t \geq nM_0 - 1$ the right-hand side is $\frac{1}{n}$, and with $y^{(t)} \geq x = \frac{1}{n}\mathbf{1}$ this forces $y^{(t)} = x$. *Stationarity.* At $y = x$, $Tx \leq x$ (row sums at most one), so $Tx - x \leq 0 < x$ and the update returns x. *Warm start.* $y^{(0)} = x + Tx \leq 2x$, so $M_0 \leq 2/n$ and $\lceil nM_0 \rceil - 1 \leq 1$. □

Proposition B (absorption and quadratic contraction of Realization B)**.** *Let T be row-substochastic and $y_i^{(t+1)} = (1 - \alpha_i)x_i + \alpha_i S_i$ with $\alpha_i = e_i/(e_i + x_i)$, $e_i = (y_i^{(t)} - x_i)_+$ and $S = Ty^{(t)}$. (i) If $y_i^{(t)} \leq x_i$ then $y_i^{(s)} = x_i$ for all $s > t$. (ii) If $x = \xi\mathbf{1}$ and $m_t = \max_i (y_i^{(t)} - \xi)_+$, then $m_{t+1} \leq m_t^2/(\xi + m_t)$; hence $m_t \leq m_0\,\theta^t$ with $\theta = m_0/(\xi + m_0) < 1$, the convergence is quadratic once $m_t \ll \xi$, and $y^{(t)} \to x$.*

Proof. (i) $e_i = 0$ gives $\alpha_i = 0$ and $y_i^{(t+1)} = x_i$; then $e_i = 0$ again at the next step, and so on. (ii) The map $e \mapsto e/(e + \xi)$ is increasing, so $\alpha_i \leq m_t/(m_t + \xi)$. Every $y_j^{(t)} \leq \xi + m_t$, and T is row-substochastic and nonnegative, so $S_i \leq \xi + m_t$. If $S_i \leq \xi$, then $y_i^{(t+1)}$ is a convex combination of ξ and $S_i \leq \xi$ and has no excess. Otherwise $y_i^{(t+1)} - \xi = \alpha_i(S_i - \xi) \leq \frac{m_t}{m_t + \xi}\, m_t$. Taking the maximum over i gives the bound. Since $m_{t+1} \leq m_t \cdot \frac{m_t}{\xi + m_t} \leq m_t\,\theta$ (because $m_t \leq m_0$ and $m \mapsto m/(\xi + m)$ is increasing), induction gives $m_t \leq m_0\theta^t$; when $m_t \ll \xi$, $m_{t+1} \leq m_t^2/\xi$. Vertices with $y_i^{(t)} < \xi$ return to ξ at the next step by (i), so $y^{(t)} \to x$. □

Proposition C (instability of the self-consistent solution)**.** *Let $T \geq 0$ and let $y > 0$ satisfy $y = Ty - x$ and $Ty > 2x$. Then (i) $\rho(T) > 1$; (ii) the Jacobians of the A and B maps at y are $J_A = T$ and $J_B = \mathrm{diag}(x/y) + \mathrm{diag}(\mathbf{1} - x/y)T$, both nonnegative with spectral radius larger than one; (iii) for $T = \lambda W$ with W row-stochastic and $x = \xi\mathbf{1}$, $y = x/(\lambda - 1)$ is such a point if and only if $1 < \lambda < 2$, and then $\rho(J_A) = \lambda$ and $\rho(J_B) = -\lambda^2 + 3\lambda - 1 > 1$.*

Proof. (i) $(Ty)_i/y_i = 1 + x_i/y_i \geq 1 + \min_j x_j/y_j > 1$; apply Lemma 6. (ii) On the active set the A map is affine, $y \mapsto Ty - x$, with Jacobian T. For B write $F_i(y) = x_i^2/y_i + (1 - x_i/y_i)S_i(y)$ with $S = Ty$. Then $\partial F_i/\partial y_j = \delta_{ij}\big(-x_i^2/y_i^2 + x_i S_i/y_i^2\big) + (1 - x_i/y_i)T_{ij}$. At the fixed point $S_i = y_i + x_i$, so the diagonal term is $x_i(S_i - x_i)/y_i^2 = x_i/y_i$, giving J_B. Because $y = S - x > x$, both x/y and $\mathbf{1} - x/y$ are positive, so $J_B \geq 0$. Now $J_A y = Ty = y + x > y$ and $J_B y = x + (\mathbf{1} - x/y) \circ (y + x) = y + x \circ (y - x)/y > y$; Lemma 6 gives $\rho(J_A) > 1$ and $\rho(J_B) > 1$. A fixed point of a C^1 map whose Jacobian has an eigenvalue of modulus larger than one is unstable (linearised instability). (iii) $W\mathbf{1} = \mathbf{1}$, so $y = c\mathbf{1}$ solves $y = \lambda W y - \xi\mathbf{1}$ if and only if $c(\lambda - 1) = \xi$; positivity requires $\lambda > 1$, and $y > x$ (equivalently $Ty > 2x$) requires $c > \xi$, i.e. $\lambda < 2$. Then $x/y = (\lambda - 1)\mathbf{1}$ and $J_B = (\lambda - 1)I + (2 - \lambda)\lambda W$. For $a, b \geq 0$ and nonnegative W with $\rho(W) = 1$ the eigenvalues of $aI + bW$ are $a + b\mu$ with $|a + b\mu| \leq a + b$, and $a + b$ is attained at $\mu = 1$; hence $\rho(J_B) = (\lambda - 1) + (2 - \lambda)\lambda = -\lambda^2 + 3\lambda - 1$, and $\rho(J_B) - 1 = (\lambda - 1)(2 - \lambda) > 0$ on $(1, 2)$. $\rho(J_A) = \lambda\rho(W) = \lambda$. □

3.4 Realization C

Proposition D (Lehmer-mean contraction below the prior)**.** *Let $\gamma = 1$, T row-substochastic, $f(x_i, s) = (x_i^2 + s^2)/(x_i + s)$ and $y^{(0)} > 0$. (i) $f(x_i, s) \geq (2\sqrt{2} - 2)x_i$, with equality iff $s = (\sqrt{2} - 1)x_i$; f decreases in s on $[0, (\sqrt{2} - 1)x_i]$ and increases afterwards; $f(x_i, s) < x_i$ iff $0 < s < x_i$. (ii) For $X \geq \max(\max_i x_i, \max_i y_i^{(0)})$ the box $\mathscr{B} = \{(2\sqrt{2} - 2)x \leq y \leq X\mathbf{1}\}$ contains every iterate from $t = 1$, and $F(y)_i = f(x_i, (Ty)_i)$ is an ∞-norm contraction on $\mathscr{B}$, so the iteration converges to a unique fixed point y^C. (iii) At y^C, $y_i^C < x_i$ whenever $0 < S_i < x_i$, and an isolated vertex outranks any non-isolated vertex with the same prior and $S_i < x_i$.*

Proof. (i) With $r = s/x_i \geq 0$, $\varphi(r) = f/x_i = (1 + r^2)/(1 + r)$ and $\varphi'(r) = (r^2 + 2r - 1)/(1 + r)^2$, whose only nonnegative root is $r^\star = \sqrt{2} - 1$; $\varphi' < 0$ on $[0, r^\star)$ and $\varphi' > 0$ after. $\varphi(r^\star) = (1 + 3 - 2\sqrt{2})/\sqrt{2} = 2\sqrt{2} - 2$. Finally $f < x_i$ iff $x_i^2 + s^2 < x_i^2 + x_i s$ iff $s(s - x_i) < 0$ iff $0 < s < x_i$. (ii) *Invariance.* By (i), $y^{(t)} \geq (2\sqrt{2} - 2)x$ for $t \geq 1$. The Lehmer mean $L_2(a, b) = (a^2 + b^2)/(a + b)$ lies in $[\min(a, b), \max(a, b)]$: if $a \leq b$, $a(a + b) \leq a^2 + b^2 \leq b(a + b)$. Since T is row-substochastic, $S_i \leq \max_j y_j^{(t)}$, so $y_i^{(t+1)} \leq \max(x_i, \max_j y_j^{(t)}) \leq X$ by induction. *Contraction.* $\partial F_i/\partial y_j = c_i T_{ij}$ with $c_i = \partial_s f(x_i, S_i) = 1 - 2x_i^2/(x_i + S_i)^2$, which is increasing in S_i and lies in $(-1, 1)$ for $S_i > 0$. On $\mathscr{B}$, a vertex with a neighbour has $S_i \in [s_i^-, X]$ with $s_i^- = (2\sqrt{2} - 2)(Tx)_i > 0$, so $|c_i| \leq \kappa_i = \max\big(|c(s_i^-)|, |c(X)|\big) < 1$; an isolated vertex has $T_{ij} = 0$ for all j. Therefore $\|J_F(y)\|_\infty = \max_i |c_i| \sum_j T_{ij} \leq \kappa = \max_i \kappa_i < 1$ on $\mathscr{B}$. $\mathscr{B}$ is convex, so the mean-value inequality gives $\|F(y) - F(y')\|_\infty \leq \kappa \|y - y'\|_\infty$, and Banach's fixed-point theorem gives a unique fixed point in $\mathscr{B}$ attracting every iterate. Boxes are nested in X, so the fixed point is independent of X and of the start. (iii) Apply (i) to $y_i^C = f(x_i, S_i)$; an isolated vertex has $S_i = 0$ and $y_i^C = f(x_i, 0) = x_i$. □

3.5 The sign obstruction

Lemma 7 (column dominance)**.** *Let $M \geq 0$ be row-substochastic with $\rho(M) < 1$ and $G = (I - M)^{-1}$. Then $G \geq 0$ and $g_{jk} \leq g_{kk}$ for all j, k.*

Proof. $G = \sum_{m \geq 0} M^m \geq 0$ converges because $\rho(M) < 1$, and $g_{kk} \geq 1$. From $G = I + MG$, the column $v = Ge_k/g_{kk}$ satisfies $v_k = 1$ and $v_j = \sum_l M_{jl} v_l$ for $j \neq k$. Suppose $\mu = \max_j v_j > 1$ and let $A = \{j : v_j = \mu\}$, so $k \notin A$. For $j \in A$, $\mu = \sum_l M_{jl} v_l \leq \mu \sum_l M_{jl} \leq \mu$; equality throughout forces $\sum_l M_{jl} = 1$ and $v_l = \mu$ whenever $M_{jl} > 0$, i.e. every row indexed by A sums to one and is supported in A. Then the principal submatrix M_{AA} is row-stochastic, so $\rho(M_{AA}) = 1$; but $\rho(M_{AA}) \leq \rho(M) < 1$ for a principal submatrix of a nonnegative matrix [3], a contradiction. Hence $v \leq \mathbf{1}$. □

Proposition E (sign obstruction)**.** *(i) For $u_i = (1 - \alpha_i)x_i + \alpha_i S_i$, $\partial u_i/\partial\alpha_i = S_i - x_i$. (ii) Let T be row-substochastic, $\alpha \in [0, 1)^n$, $D = \mathrm{diag}(\alpha)$, $y(\alpha) = (I - DT)^{-1}(I - D)x$ and $S = Ty$. Then $\partial y/\partial\alpha_k = (S_k - x_k)Ge_k$ with $G = (I - DT)^{-1}$, and $\partial(y_k - y_j)/\partial\alpha_k = (S_k - x_k)(g_{kk} - g_{jk})$ has the sign of $S_k - x_k$ or vanishes. (iii) For $T = EW$ and $x = \frac{1}{n}\mathbf{1}$, $(EWx)_i = P_{\mathrm{esc}}(i)/n < x_i$ whenever $P_{\mathrm{esc}}(i) < 1$.*

Proof. (i) Differentiate. (ii) DT is nonnegative with row sums at most $\max_k \alpha_k < 1$, so $\rho(DT) < 1$, $I - DT$ is invertible and y is well defined. Differentiating $y = (I - D)x + DTy$ with respect to α_k gives $\partial y = -x_k e_k +$

$(Ty)_k e_k + DT\,\partial y$, i.e. $(I - DT)\partial y = (S_k - x_k)e_k$. Lemma 7 with $M = DT$ gives $g_{kk} - g_{jk} \geq 0$. (iii) For a non-isolated vertex $(W\mathbf{1})_i = 1$, so $(EW\frac{1}{n}\mathbf{1})_i = P_{\text{esc}}(i)/n$; an isolated vertex has $S_i = 0 < x_i$. □

3.6 The pull variant

Proposition F (degeneracy under uniform priors)**.** *Let W be row-stochastic without isolated vertices, x uniform and $S = Wy$. Every iterate of Realizations A, B and C from $x + Wx$, on the raw, L_1 or excess scale, is a multiple of x.*

Proof. $Wx = x$ because $W\mathbf{1} = \mathbf{1}$ and $x \propto \mathbf{1}$. If $y = cx$ ($c > 0$), then $S = cx$ and the updates are $\max(1, c-1)x$ (A), $\big((1-a) + ac\big)x$ with $a = (1 - 1/c)_+$ (B) and $\frac{1+\gamma c^2}{1+\gamma c}x$ (C). The raw scale is the identity, the L_1 scale divides by a constant, and the excess scale maps cx to $x + \gamma x = (1+\gamma)x$ if $c > 1$ and to x otherwise (the implementation returns x when the excess vanishes). The warm start is $2x$; induction completes the proof. All scores tie, and a constant score vector has $\rho = 0$ by convention. □

3.7 Prior dispersion

Proposition $2'$ (log-dispersion identity)**.** *Let $z_i = y_i - x_i > 0$, $r_i = z_i/x_i$, and let $\log z$, $\log x$ have standard deviations $\sigma_z, \sigma_x > 0$ and correlation ϕ over the vertices, with $\log r$ non-constant. Then* $\operatorname{corr}(\log r, \log z) = (\sigma_z - \phi\sigma_x)/s$ *and* $\operatorname{corr}(\log r, -\log x) = (\sigma_x - \phi\sigma_z)/s$ *with $s = \sqrt{\sigma_z^2 + \sigma_x^2 - 2\phi\sigma_z\sigma_x}$; as $\sigma_x/\sigma_z \to \infty$ they tend to $-\phi$ and* 1.

Proof. $\log r = \log z - \log x$. Bilinearity of covariance gives $\operatorname{Var}(\log r) = \sigma_z^2 + \sigma_x^2 - 2\phi\sigma_z\sigma_x = s^2 > 0$, $\operatorname{Cov}(\log r, \log z) = \sigma_z^2 - \phi\sigma_z\sigma_x$ and $\operatorname{Cov}(\log r, -\log x) = \sigma_x^2 - \phi\sigma_z\sigma_x$. Dividing by $s\sigma_z$ and $s\sigma_x$ gives the identities. With $q = \sigma_z/\sigma_x \to 0$: $(q - \phi)/\sqrt{q^2 + 1 - 2\phi q} \to -\phi$ and $(1 - \phi q)/\sqrt{q^2 + 1 - 2\phi q} \to 1$. □

The manuscripts' Proposition 2. Part (1) states that under the uniform prior $\alpha_i = 1 - 1/(ny_i)$ is a strictly monotone, shift-invariant function of y_i. Monotonicity holds; shift-invariance does not, since replacing y by $y + c\mathbf{1}$ changes α_i by $\frac{1}{n}\big(\frac{1}{y_i} - \frac{1}{y_i + c}\big)$, which depends on i. Monotonicity does not imply a ranking advantage, because by Proposition E the effect of a larger α_i has the sign of $S_i - x_i$. Part (2), $|\partial_{x_i}((y_i - x_i)/x_i)| = y_i/x_i^2$, is a correct derivative; Proposition $2'$ describes what happens to the ranking content of the ratio under smooth heterogeneous priors.

4 Module-by-module walkthrough

The descriptions below follow the source files; function names are given with their main arguments, and $\text{nnz} = 2m$ is the number of stored entries of a symmetric sparse operator.

4.1 `prop_damping_lib.py`

Instances and priors. `Instance` is a dataclass holding `name`, `n`, `edges` ($i < j$, unique), `p`, `x`, `meta`, with a lazily built operator cache (`ops()`) and `with_prior(x)`, which shares the graph and the probabilities. `topology(kind, n, seed)` generates the nine synthetic families through NetworkX (Erdős–Rényi $G(n,m)$ with mean degree 5, Barabási–Albert with $m = 2$, connected Watts–Strogatz with $k = 6$ and rewiring 0.1, square and 1:4 rectangular grids, four-block SBM with expected internal and external degrees 5 and 1, random labelled trees, barbells) and builds the core–periphery family directly (15% core with internal density 0.5, one or two core contacts per periphery vertex, plus random periphery edges). `PROB_RANGES` defines R_1–R_5. `make_prior(kind, n, seed)` produces the uniform, Dirichlet(θ) (normalised Gamma draws), log-normal(σ) and softened seed priors, floored at 10^{-12} of the maximum before normalisation. `make_synthetic` assembles an instance, seeding topology and probabilities independently of the prior. For real networks, `load_npz_graph` reads the sparse adjacency from a `.npz` file with `allow_pickle=False` and symmetrises it, `snowball` takes a breadth-first sample of the requested size from a random vertex of the

largest component, `subgraph_edges` extracts the induced edges, and `real_probs` implements the constant, uniform-interval, weighted-cascade and capped weighted-cascade schemes.

Operators. `escape_probability` computes

$$P_{\mathrm{esc}}(i) = -\mathrm{expm1}\Big(\sum_j \log 1p(-p_{ij})\Big)$$

by two `bincount` passes over the edge list, with p clipped below $1-10^{-12}$ inside the logarithms. The function `build_operators` returns P_{esc}, $\bar{P}_{\mathrm{esc}}$, degrees, expected degrees and the CSR matrices W (weights $-\log(1-p_{ij})$, rows normalised), EW, $W^{\top}$, P and EP. `rule_alpha(Pbar, family)` evaluates the calibrated rules of [1] for the pull $(0.502+0.558\,\bar{P}_{\mathrm{esc}})$ and push $(0.241+0.859\,\bar{P}_{\mathrm{esc}})$ operators, clipped to $[0.4,1]$.

Ground truth. `mc_reach(inst, M, seed)` implements Lemma 3: worlds are drawn in batches of size $b=\min(M,\lfloor N_{\max}/n\rfloor,\lfloor E_{\max}/m\rfloor)$ (defaults $4\cdot10^5$ vertices and $4\cdot10^6$ edges), the kept edges of the batch are offset into one block-diagonal sparse matrix, a single `connected_components` call labels all worlds, and the prior mass of each component is accumulated by `bincount`. It optionally returns the second moment. `exact_reach` enumerates all 2^m worlds ($m\le 20$) and is used only in the tests.

Metrics. `ranks_of` returns average ranks after merging scores equal to within a relative tolerance of 10^{-12} (so that round-off does not break ties); `corr_ranks` computes Pearson correlations of rank vectors, batched over columns and returning zero for constant predictions; `spearman` combines the two. `full_metrics` adds Kendall's τ-b, the overlap of the top $\lceil 0.1n\rceil$ vertices and NDCG@10% with min–max normalised relevance.

Diffusions and message passing. `diffuse(T, x, alpha, y0)` implements Lemma 1 for scalar or per-vertex α with the lazy iteration and an L_1 stopping rule (10^{-12}); `diffuse_batch` runs many damping vectors at once as the columns of a dense $n\times K$ matrix, which is what makes the scalar oracle and the per-node oracle affordable. `knz_mp(inst, iters)` runs the message passing of [2] in membership form on the $2m$ directed messages in log space; `iters=0` returns P_{esc} (Lemma 4) and `iters=None` runs to convergence.

The principle. `theorem1_roots` returns both branches of Theorem 1. `rescale(y, x, scale, gamma)` implements the raw, L_1 and excess scales. `realization_step(kind, y, S, x, ...)` applies one update of A, B or C; the optional `alpha_transform` replaces α by its mean or a permutation (the matched controls), and `xr` supplies the regularised denominator. `run_realization` runs Mode (a) from the warm start (or a given start), records Spearman ρ, the trivial and active fractions and the α statistics at checkpoints, and, if the raw scale overflows on an expansive operator, keeps the last finite state (its ranking has converged to the dominant direction). `anchored_alpha(q, alpha0, eta)` implements $\mathrm{AP}(\eta)$ with the safeguarded Newton iteration of Lemma 2; `power_alpha` is the unanchored law $\alpha/(1-\alpha)=\gamma q^{\eta}$; `polish_ap` runs Mode (b) with the anchored law, either with α fixed from the base or self-consistently.

Oracles. `scalar_sweep` runs the batched diffusion for the 40 values of `ALPHA_GRID`. `pernode_oracle(T, x, fit_ranks, alpha_init)` performs coordinate ascent: vertices are visited in random order, every other value of the 14-value grid is evaluated for the current coordinate in one batched diffusion warm-started from the current solution, and the best is accepted if it improves the Spearman fit by more than 10^{-12}; it stops after three sweeps, a sweep without improvement or the time limit.

4.2 `build_panels.py`

`spec_list(panel)` enumerates the specifications of each panel (Section 5.1 of the paper); `make_instance` builds one instance (synthetic, classic real network, or snowball sample with `data_real/*.npz`); `build_one` computes the two 2,000-world ground-truth samples with seeds derived from the instance name, the second moment, $\bar{P}_{\mathrm{esc}}$, prior statistics and the split-sample reference, and writes `data/<panel>/NNNN.npz`;

`load_instance` reads it back. The stage runs with a pool of two worker processes and skips instances that already exist.

4.3 `experiments.py`

`Recorder` accumulates result rows and computes, for a base and a polished vector, the score and rank blends for every β. `ap_run` iterates the anchored (or unanchored) law jointly with the diffusion, recording finite step counts and continuing with lazy sweeps to the fixed point for $T = \infty$. `polish_fixed` runs K sweeps with a fixed α. `excess_embed` implements Lemma 5. `baselines_and_oracles` computes all references, both oracles (fitted on y_B^*) with the per-node diagnostics (correlations of α^* with candidate features), and the true-y^* variants. `grid_instance` runs the full development search: every realization on every operator and scale with checkpoints $T \in \{1,2,3,5,10,20,50,200\}$ (and two extra γ for C), the anchored law for eight values of η, the unanchored law, anchored polishing of both bases with fixed and self-consistent α, $K \in \{1,2,5,\infty\}$ and all blends, a permutation control, and realization polishing from excess embeddings with $\gamma \in \{1,4,16\}$. `selected_instance` evaluates the frozen configurations (with the τ-regularised variants on the prior panel). `run_one` handles one instance (time-capping the per-node oracle and running it at $n = 900$ only on grid, rectangular-grid, Erdős–Rényi and Barabási–Albert instances) and writes the per-instance files; `main(stage, panels, procs)` runs a stage over panels with `procs` worker processes (default two) and skips instances whose results exist (resumable).

4.4 `select_configs.py`

Re-derives every frozen configuration from `results/grid_dev` and `results/controls_dev.csv.gz` with the rules listed in its header (best per family, best per prior type, the manuscripts' nominal settings, and the matched controls of the best polisher), and compares the result with `results/selected.json` entry by entry. In the delivered results all 21 entries are consistent: twenty match exactly and one (`PB_mp2_eta0`) differs only in a mode that is irrelevant at $\eta = 0$, where the fixed and self-consistent polishes coincide. The frozen file is never overwritten; `--write` saves the derived selection elsewhere.

4.5 Analysis, controls and diagnostics

`analysis.py`: `load_stage(stage, panel)` returns the concatenated result rows, the instance metadata and the per-instance diagnostics; `config_means` ranks configurations by mean ρ; `paired(a, b)` returns the mean difference, its 20,000-resample percentile-bootstrap interval, the Wilcoxon signed-rank p-value on untied pairs and W/T/L; `holm` implements the step-down Holm adjustment. `controls.py` sweeps the best realization polisher with the law C and its replacements (mean, permutation, constant $\alpha \in \{0.5, 0.9, 1\}$) on the development panel. `mf_controls.py` re-runs every frozen realization configuration with its law, its mean-field twin and its permutation on dev, test, real and scale. `sign_condition.py` computes, at the rule's fixed point, the fraction of vertices with $S_i > x_i$ and relates it to the per-node oracle and to the true-y^* law. `prop2_stats.py` computes the Proposition 2 statistics on ground truth: $\kappa(x)$, the range and saturation of the ratio for several τ, and its rank correlations with y^* and with $1/x$. `explore_rawP.py` evaluates the exploratory raw-P propagation. `mc_baseline.py` ranks by independent k-world estimates. `bench_runtime.py` times every method in a single process (median of three runs), including the end-to-end pipelines. `paper_extras.py` records the per-step dynamics traces (`traces`), the operator complementarity (`complement`) and the size sweep (`sweep`, sizes 100 to about 2000 on four families, ensemble and Monte Carlo timed in one process).

4.6 Reporting

`make_checkpoint_tables.py` and `write_checkpoint_report.py` produce the Checkpoint-1 tables and report. `make_paper_assets.py` loads every result file, computes the panel summaries, Holm corrections, attribution statistics, the size-sweep analysis (equal-cost interpolation in $\log k$ and crossover sizes), writes `paper/generated/macros.tex` (one `\newcommand` per number), `tables_en.tex` and `tables_gr.tex`

(every table as a macro), and calls `paper_figures.py`, which draws the five figures in both languages with a validated colour palette and FreeSerif text. No number in the papers is typed by hand.

4.7 Tests

`tests/test_foundations.py` checks Theorem 1 on random inputs (largest residual 3.2×10^{-12}), the Monte Carlo estimator against exact enumeration (largest $|z|$-score 2.6), Lemma 4, the agreement of lazy and plain diffusion and the fixed-point equation, the bipartite case, and the collapse of Proposition A after one step. `tests/test_dynamics.py` checks Propositions A–E numerically: collapse and absorption, the spectral radii of Proposition C for $\lambda \in \{1.2, 1.5, 1.9\}$ and the departure of perturbed fixed points, the uniqueness of the Realization C fixed point from three starts, and the sign rule by finite differences.

5 Complexity analysis

Let n be the number of vertices, m the number of edges, and $\mathrm{nnz} = 2m$. Table 3 lists the cost of every component; all propagation steps are sparse matrix–vector products and therefore cost $O(n+m)$ per sweep.

Table 3: Time and memory of the components (I: number of iterations to tolerance; K: sweeps; M: worlds; b: batch size).

Component	Time	Memory
Operators (P_{esc}, W, EW, $W^\top$, P, EP)	$O(n+m)$	$O(n+m)$
Damped diffusion (scalar or per-vertex α)	$O(I(n+m))$	$O(n+m)$
Batched diffusion of K damping vectors	$O(IK(n+m))$	$O(Kn+m)$
Realization A/B/C, T steps	$O(T(n+m))$	$O(n+m)$
Anchored law: one evaluation of α	$O(nN_{\mathrm{Newton}})$, $N_{\mathrm{Newton}} \le 60$	$O(n)$
AP(η) to the fixed point	$O(I(n+m+nN_{\mathrm{Newton}}))$	$O(n+m)$
Mode (b) polish (K sweeps) + blend	$O(K(n+m)+n\log n)$	$O(n+m)$
Message passing, k iterations	$O(km)$	$O(m)$
Monte Carlo, M worlds	$O(M(n+m))$ expected (linear-time components per batch)	$O(b(n+m))$
Exact enumeration (tests)	$O(2^m(n+m))$	$O(n+m)$
Spearman / Kendall / top-k / NDCG	$O(n\log n)$	$O(n)$
Scalar oracle (40 values)	$O(40I(n+m))$ batched	$O(40n+m)$
Per-node oracle	$O(Sn\,13I(n+m))$ for $S \le 3$ sweeps (time-capped)	$O(13n+m)$

Per-sweep cost and the dual-operator ensemble. One sweep of any diffusion or realization costs one sparse product with $\mathrm{nnz} = 2m$ entries plus $O(n)$ vector work, i.e. $O(|E|)$ for connected graphs. The dual-operator ensemble of the paper costs the operator construction, I lazy sweeps of the calibrated diffusion, $K = 5$ sweeps on P and two rankings: $O((I+K)(n+m)+n\log n)$.

Monte Carlo. Each world requires drawing m Bernoulli variables and labelling components in $O(n+m)$; batching does not change the asymptotic cost but replaces M library calls by $\lceil M/b \rceil$, which is why it is fast in practice (Section 6.9 of the paper). The equal-cost comparison of the paper therefore compares an $O((I+K)(n+m))$ deterministic method with an $O(k(n+m))$ sampler; the crossover occurs where k worlds cost the same as $I+K$ sweeps plus overheads, which depends on I (hence on the family and the damping) and on constant factors.

Per-node oracle. The oracle dominates the experimental budget: each coordinate evaluates 13 alternatives in one batched diffusion, so a sweep costs $O(13\,nI(n+m))$, quadratic in n for sparse graphs. This is why it is time-capped and run at $n = 900$ only on a subset of families.

Measured stage times. With 2 worker processes, building all panels took 36 s; the development and test grids took 369 s and 421 s; the confirmatory stage took 178 s (test), 242 s (prior), 689 s (real) and 1244 s (scale), the last two dominated by the per-node oracle.

6 Reproduction guide

6.1 Environment

The results were produced with Python 3.13.17, NumPy 2.5.3, SciPy 1.18.1, pandas 3.0.5, NetworkX 3.7 and Matplotlib 3.11.2 on a machine with 2 cores. The documents are compiled with XeLaTeX (TeX Live 2023) and use the FreeSerif, FreeSans and DejaVu Sans Mono fonts and the `mathptmx` mathematics.

```
pip install numpy scipy pandas networkx matplotlib tabulate
```

6.2 Step by step

All commands are run from the package root. Steps 3–5 can be skipped, because their outputs are shipped in `data/` and `results/`.

1. *Check the foundations.* Both test scripts print their checks and end with "ALL … CHECKS PASSED".

   ```
   python tests/test_foundations.py
   python tests/test_dynamics.py
   ```

2. *Real-network sources* (only needed to rebuild the real panel). Download `cora.npz`, `citeseer.npz`, `amazon_electronics_photo.npz` and `ms_academic_cs.npz` from the repository of [4] into `data_real/`. The sampled subgraphs used in the paper are already stored in `data/real/`.

3. *Panels and ground truth* (about 36 s for all five panels).

   ```
   python build_panels.py
   ```

4. *Development and test search* (about 369 s and 421 s), and the development control grid.

   ```
   python experiments.py grid dev,test
   python controls.py dev
   ```

5. *Selection.* The first command verifies that the frozen file follows from the development results by the stated rules; the second must print the digest recorded in `results/selected.sha256`.

   ```
   python select_configs.py
   sha256sum results/selected.json
   ```

6. *Held-out evaluation* (about 178, 242, 689 and 1244 s).

   ```
   python experiments.py confirm test,prior,real,scale
   ```

7. *Controls and diagnostics.*

   ```
   python mf_controls.py dev,test,real,scale
   python sign_condition.py
   python prop2_stats.py
   python explore_rawP.py dev,test
   python explore_rawP.py real,scale
   python mc_baseline.py test,real,scale
   ```

8. *Timings and supplementary computations.* Timings depend on the machine; every other output is deterministic.

```
python bench_runtime.py
python paper_extras.py traces
python paper_extras.py complement
python paper_extras.py sweep
```

9. *Tables, macros and figures* (add `--no-figures` to the last command to skip the figures).

```
python make_checkpoint_tables.py
python make_paper_assets.py
```

10. *Documents.* Each paper is compiled three times (until its cross-references are stable), and then each documentation file three times (for the table of contents; the documentation takes the numbers of the paper's tables, figures and sections from the paper's auxiliary files).

```
cd paper
xelatex paper_en.tex           # three times
xelatex paper_gr.tex           # three times
cd ..
xelatex DOCUMENTATION_EN.tex   # three times
xelatex DOCUMENTATION_GR.tex   # three times
```

6.3 From the paper to the code

Table 4 maps every table and figure of the paper to its source data and to the function that produces it (in `make_paper_assets.py` or `paper_figures.py`).

Table 4: From the paper to the code.

Paper element	Generator	Source data
Every inline number	`build_macros`	all files below
Table 1 (panels)	`tab_panels`	`data/<panel>/`
Table 2 (replication)	`tab_replication`	`results/tables/T2_replication_of_rule.csv` (from `results/grid_dev`, `results/grid_test`)
Table 3 (headroom)	`tab_headroom`	`results/tables/T3a_pernode_headroom_by_family.csv`, `T3b_pernode_headroom_by_regime.csv`
Table 4 (confirmatory)	`tab_confirm`	`results/confirm_test/`
Table 5 (prior panel)	`tab_prior`	`results/tables/T9a_prior_panel_methods.csv`, `T9b_prior_panel_diff_vs_rule.csv` (from `results/confirm_prior`)
Table 6 (attribution)	`tab_attribution`	`results/mfcontrols_*.csv.gz`
Table 7 (transfer)	`tab_transfer`	`results/confirm_real/`, `results/confirm_scale/`
Table 8 (secondary)	`tab_secondary`	per-instance JSON diagnostics
Table 9 (complementarity)	`tab_complement`	`results/complementarity.csv`
Table 10 (cost)	`tab_cost`	`results/tables/T12_accuracy_vs_cost.csv` (from `results/runtime.csv` and the confirmatory results)

Table 4: From the paper to the code (continued).

Paper element	Generator	Source data
Table 11 (size sweep)	`tab_sweep`	`results/size_sweep.csv`
Tables 12–13	`tab_configs`, `tab_dynamics`	`results/selected.json`, `results/tables/T13_dynamics_dev.csv`
Figure 1	`fig1`	`results/traces_dev.csv.gz`, `results/grid_dev`, `results/tests_dynamics.json`
Figure 2	`fig2`	`results/sign_condition.csv`, `results/tables/T5_anchored_eta_curve.csv`
Figure 3	`fig3`	`results/confirm_prior/`, `results/prop2_stats.csv`
Figure 4	`fig4`	`results/confirm_*`, `results/mfcontrols_*.csv.gz`
Figure 5	`fig5`	`results/tables/T12_accuracy_vs_cost.csv`, `results/size_sweep.csv`

7 Extending the package

A new per-node law is added as a branch of `realization_step` (or as a function of $q = y/x$ for the anchored family), registered in `grid_instance`, selected on the development panel and frozen by adding its entry to a new selection file before any held-out run; `mf_controls.py` then provides its matched controls automatically. A new operator is added to `build_operators` and to `OPS`. A new panel needs a branch in `spec_list`; the ground truth, the confirmatory stage and the reporting work unchanged. Keeping the selection on the development panel and the evaluation on frozen configurations preserves the validity of the held-out tests.

References

[1] I. Konstantellos. Damping from the escape probability: a free calibration rule for diffusion-based reachability ranking in uncertain graphs. Computer Engineering and Informatics Department, University of Patras, 2026. `https://zenodo.org/records/23211440`.

[2] B. Karrer, M. E. J. Newman, L. Zdeborová. Percolation on sparse networks. *Physical Review Letters*, 113:208702, 2014.

[3] R. A. Horn, C. R. Johnson. *Matrix Analysis*, 2nd edition. Cambridge University Press, 2013.

[4] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann. Pitfalls of graph neural network evaluation. Relational Representation Learning Workshop, NeurIPS 2018. arXiv:1811.05868.